Sherm Quanty

RegDet V1.1 -- Study Notes

A 2-hour Nifty regime detector: architecture, the Hidden Markov Model, every chart in the master notebook explained, the six audited bugs, and the shipped configuration.

Working notes -- not for redistribution as a finished report. All real-data numbers are from the user's own Kaggle run (final_regdet_v9.ipynb, real yfinance, 2894 2h bars, 2023-08-21 -> 2026-07-31). Where a number is illustrative-only (sandbox synthetic or GitHub daily data), it is labelled as such.

1. What RegDet is

RegDet labels every 2-hour Nifty bar with one of five regimes:

Label	Meaning
H_BULL	High-conviction bullish -- strong AND clean upward trend
L_BULL	Low-conviction bullish -- upward, but weak or choppy
SIDEWAYS	No reliable direction
L_BEAR	Low-conviction bearish
H_BEAR	High-conviction bearish -- strong AND clean downward trend

The label is built from **two independent decisions** that get combined:

- **Direction** (bull / sideways / bear) -- decided by a Hidden Markov Model (HMM), optionally blended with a simple per-bar momentum score.
- **Intensity** (High vs Low conviction) -- decided entirely by price statistics (trend strength and trend efficiency). **No HMM is involved in this axis at all.** This is a load-bearing fact discovered mid-project: the H-vs-L split works using zero machine learning, which is why changing the HMM's weight in the direction blend barely moves occupancy.

Nothing here is predictive by design intent alone -- every claim of "this label predicts returns" is checked against real forward-return data with proper statistics (Section 6), and the checks do not always pass cleanly. That is reported, not hidden.

2. The Hidden Markov Model, in full

2.1 The generative story

An HMM assumes two layers. A **hidden** layer: at every 2h bar, the market is secretly in one of N states (N=5 in the shipped config), and the state at bar t depends only on the state at bar t-1 (the **Markov property** -- no longer memory than that). An **observed** layer: given the hidden state, the 9 engineered features (Section 3) are drawn from that state's own probability distribution. You only ever see the observed layer; the entire exercise is inferring the hidden layer from it.

2.2 The three parameter blocks

Parameter	Code name	Meaning
π (pi)	startprob_	Probability the very first bar started in each state
A	transmat_	N×N matrix; A[i,j] = P(move from state i to state j next bar). Large diagonal = "sticky" states = persistence
μ, Σ	means_, covars_	Each state's own mean vector and covariance over the 9 features (a multivariate Gaussian per state -- hence

2.3 Covariance type: diag vs full -- and why it drives model size

full covariance lets every pair of features correlate within a state (e.g. "in this state, high momentum tends to come with high volatility") -- a 9×9 symmetric matrix per state, 45 numbers just for the spread of one state. **diag** assumes each feature varies independently within a state -- 9 numbers instead of 45. This single choice is why the three candidate configs have wildly different parameter counts:

Config	N states	Cov type	Features	Free params	Rows/param (fold 1)
V1.0 (prod)	5	full	9	294	4.9 -- unhealthy, rule of thumb wants >=10
A: lean-cov (SHIPPED)	5	diag	9	114	12.5 -- healthy
B: lean-feat	5	diag	5	74	19.3 -- healthiest, fewer features

Fewer parameters need less data to estimate reliably. This is the argument for 'A: lean-cov' on **structural** grounds -- not because it won a performance contest (Section 7 shows that contest is undecidable at this sample size).

2.4 Fitting: EM / Baum-Welch, step by step

Fitting means finding, purely from data, the values of π, A, μ, Σ that best explain what was observed. This is done by Expectation-Maximization (EM), which for an HMM specifically is called **Baum-Welch**. It alternates two steps until the fit stops improving:

- **E-step ("guess"):** given the current π/A/μ/Σ, compute the probability that bar t was in each state, for every bar in the training window. This needs the FULL forward-backward pass (both directions in time) -- allowed here because fitting is only ever done on the past training slice, never live.
- **M-step ("update"):** given those probabilities, recompute each state's typical feature values as a probability-weighted average across all training bars, and recompute A from how often transitions between states were implied.

Each round is guaranteed to never make the fit worse, measured by log-likelihood (roughly: "how surprised would this model be by the data it just saw"). It stops when that stops improving, or after a hard cap of 2000 iterations (real fits typically converge in 75-240).

The catch: EM only climbs to a LOCAL best answer -- like feeling your way uphill in fog. Where you start (the random seed) can determine which peak you land on. This project measured this directly: refitting the identical model with different seeds produced two distinct "basins" -- ~98% label agreement within a basin, ~63% across basins. Nested, highly-correlated momentum features (mom_1d/3d/5d overlap heavily) are the suspected cause -- redundant information creates multiple comparably-good peaks for EM to land on.

2.5 The forward algorithm -- causality made mathematical

Define α_t(i) = P(state at t = i AND everything observed up to t | model). It has a clean recursion:

```
α_t(j) = [ Σ_i α_(t-1)(i) · A(i,j) ] × emission_j(observation at t)
```

In words: "the chance of being in state j now, predicted forward from every state I could have been in last bar, times how well state j's distribution explains what was just observed." Normalize across states at each step and you get P(state_t = i | everything up to and including t) -- the **filtered** posterior. Every term only touches bars ≤ t. This is exactly the function `_filtered_posteriors` in the generator, and it is why the engine is causal by construction rather than by discipline.

2.6 Smoothing -- the look-ahead trap

A mirror-image backward recursion β runs from the END of the sequence backward. Combining α and β gives γ_t(i) = P(state at t = i **|** THE ENTIRE SEQUENCE, future included). This is what a library's default `predict_proba` gives you, and it is strictly more accurate *as a description of the past* -- but using it to generate a live label is look-ahead, since bar t's label would then depend on bars after t. Banned everywhere in this project except for grading already-made decisions in hindsight.

2.7 Numerical underflow -- two implementations, proven equal

α is a running product of many probabilities, so after a few hundred bars it underflows to zero in ordinary floating point. Two fixes exist in the generator: normalize at every step in linear space (divide by the running sum -- the fast path), or work entirely in log-space with `logsumexp` (the reference / fallback path). Both were verified to agree to **8.9×10■^{1}■** -- machine precision -- confirming the fast path is a pure speed optimisation, not a different computation.

3. The nine features

All computed causally (rolling/trailing windows only, never using future bars). The momentum ladder stays fast (1/3/5 days); the context window (drawdown, distance-from-MA) was widened from 6.7 to 12 days -- see Section 8.

Feature	What it measures	Window (shipped)
ret_2h	Return over the current bar	1 bar
mom_1d / mom_3d / mom_5d	Cumulative return over the last 1 / 3 / 5 trading days	3 / 9 / 15 bars
vol_2h	Recent realised volatility (rolling std of returns)	10 bars (~3.3d)
vol_expansion	Is volatility itself rising? (fast vol ÷ slow vol)	5 / 20 bars
vix_chg	Day-over-day change in India VIX	1 bar
drawdown	How far price has fallen from its recent peak	36 bars = 12 days
dist_ma	Distance of price from its own recent moving average	36 bars = 12 days

India VIX is the market's own real-time estimate of how much it expects itself to swing, derived from options prices -- it is one of the only genuinely *external* pieces of information in the feature set; everything else is a function of price itself.

4. Turning the HMM's output into bull / bear / sideways

4.1 State scoring and bucketing

Each hidden state gets scored as bullish or bearish by combining its typical feature values with sign/weight rules (a state whose typical momentum features are all positive scores bullish). States are then bucketed into bear/side/bull by rank -- at N=5 this is 2 bear / 1 side / 2 bull states.

Bug found and fixed: the original bucketing rule handed every LEFTOVER state to the bull bucket via floor division (at N=6: 2 bear / 1 side / 3 bull -- a structural bullish bias with no basis in the data). Fixed to keep bear and bull counts equal by construction, with the remainder absorbed by the side bucket. The shipped N=5 config is bit-identical before and after this fix; only N=4/6/9 configs were affected.

4.2 Ensemble averaging across seeds

The model's state PROBABILITY at bar t (not a hard guess) is summed within each bucket, giving three numbers per bar -- bull mass, side mass, bear mass -- that add to 1. Because EM can land in different local optima, several models are fit with different random seeds (ENSEMBLE_K, shipped =4) and their bucket masses are AVERAGED. This only works because the masses are **permutation-invariant**: model A's "state 3" and model B's "state 1" are arbitrary labels, but "probability mass sitting in bullish states" means the same thing regardless of numbering.

4.3 BAR_DIR_WEIGHT -- blending HMM with per-bar momentum

```
bull_mass = (1 − w)·HMM_mass + w·per_bar_momentum_score
```

w=0.0 is 100% HMM; w=1.0 is 0% HMM (pure momentum, no model at all). A large part of this project was determining whether w matters. The finding, from a controlled 5-arm sweep on real data sharing ONE fit: occupancy and switch rate barely move between w=0.0 and w=0.75 (SIDEWAYS 35.7% to 38.0%; H_BULL, H_BEAR within ~2pp everywhere). A frozen pre-declared protocol later showed the difference in predictive skill across w values is **smaller than the noise from merely reseeding the same w** (between-w spread 0.297 vs seed spread 0.615 -- reported as UNMEASURABLE, no winner named). The two axes that DO move -- predictive skill vs. "looks right" / descriptive fidelity -- were shown to be in direct tension and NOT combinable into one score by design.

4.4 CONF_L -- the low-confidence override

If the winning bucket's mass does not clear CONF_L (shipped =0.50), the bar is forced to SIDEWAYS regardless of which bucket won. Investigated directly as a hypothesis for excess SIDEWAYS occupancy and REFUTED on real daily data: it fires on only 0.07% of bars (median winning mass is 0.99). The true driver of high SIDEWAYS turned out to be a DIFFERENT mechanism -- see Section 9.

4.5 CONFIRM_BARS -- anti-flicker delay

A new label must hold for CONFIRM_BARS (=2) consecutive bars before it is actually emitted, so single-bar noise cannot flip the regime back and forth. Measured effect: median run length improved from 3.0 to 4.0 bars, total run count fell 659 → 554, while a genuinely strong sustained move was labelled identically either way (13 runs, unchanged).

5. Intensity: High vs Low conviction -- zero HMM

Separately from direction, every bar gets two price statistics:

- **trend_z** -- how extreme the current momentum is versus its own recent history, standardized (a z-score). Uses TREND_FEATURE = mom_3d.
- **trend_efficiency** -- net price displacement over a window ÷ total distance travelled in that window. 1.0 means a straight-line move; near 0 means pure back-and-forth chop with no net progress. Measured over EFF_WIN (9 bars = 3 days).

A bull or bear bar only escalates to H_BULL / H_BEAR if BOTH |trend_z| and trend_efficiency clear their thresholds (Z_HI=0.5, EFF_HI=0.35) -- strong AND clean, not just strong. Everything else stays L. Gate **hysteresis** uses slightly looser exit thresholds than entry (Z_HI_EXIT=0.35, EFF_HI_EXIT=0.25) so a bar sitting exactly on the boundary does not flicker in and out of H every bar. A bug found here: nothing previously asserted exit ≤ entry, so an inverted config could silently turn hysteresis into a no-op -- an explicit guard assert was added.

This entire axis is **100% price statistics -- no HMM anywhere in it.** This is why the H-vs-L split does not move when BAR_DIR_WEIGHT changes, and it means any answer to "how much HMM is in RegDet" only ever describes HALF the detector.

6. Validating the labels -- the six-family scorecard

6.1 Why forward returns need careful statistics

A "9-bar forward return" computed at every bar overlaps its neighbours by 8 bars -- not independent samples even though there are thousands of rows. Two corrections are applied before anything is called significant:

- **HAC / Newey-West standard error** -- explicitly measures how much a bar's forward return correlates with its neighbours (out to a lag = the horizon) and inflates the uncertainty estimate before computing a t-statistic (the `hac_t` column). n_eff = n/horizon expresses the same idea bluntly: 1000 raw bars at a 9-bar horizon are worth roughly 111 truly independent observations.
- **Cohen's d** -- the gap between two groups' means divided by their pooled spread. Reported alongside significance so a huge sample can't make a tiny, practically meaningless effect look impressive.

6.2 Six independent families -- deliberately no combined score

Family	Question it answers
A. DISCRIMINATION	Do labels predict future returns, correctly ordered and signed?
B. PERSISTENCE	Do regimes hold for a sensible run length, not flicker?
C. CALIBRATION	Does the model's own confidence track actually being right more?
D. INTENSITY	Do H bars really behave differently from L bars?
E. COVERAGE	Does every label appear a reasonable, non-degenerate amount?
F. ECONOMIC	Does a naive long/flat rule beat buy-and-hold? (noisiest family -- read last)

No single score is computed by design -- collapsing six different failure modes into one number would let a detector ace four families while hiding a break in a fifth.

6.3 Signed grading -- a real bug, found and fixed

The original grading took the ABSOLUTE VALUE of a statistic. A HAC t of +1.37 on L_BEAR looks fine at a glance -- but it means bear-labelled bars were followed by GAINS, i.e. the label was backwards for that row, and abs() scored it a WARN instead of a fail. Fixed: bull labels now require a positive effect, bear labels a negative one, SIDEWAYS has no directional expectation and grades NA, and a materially backwards result gets **FAIL(ANTI)** -- explicitly worse than no signal, not just "unproven". The fix made the reported tally WORSE (as it should) and was deliberately NOT allowed to move any threshold to compensate.

6.4 A known open inconsistency: L_BEAR at the 15-bar horizon

On the primary real run, L_BEAR shows the HIGHEST mean 15-bar forward return of any label (+0.31%, vs H_BULL's +0.01%) -- backwards at face value. The per-fold breakdown shows this holds in only 2 of 4 folds and is explicitly reported as MIXED / INCONCLUSIVE rather than built into a narrative -- with only 4 folds, a single unusual window (2026's whipsaw-heavy fold) can swing a pooled average without representing a repeatable property of the label. Open item, not resolved.

7. Picking a configuration -- and why that ranking is undecidable

Three candidate configs are compared via **walk-forward evaluation**: fit on a leading chunk of data, grade ONLY on the untouched remainder, repeat across several folds so one lucky/unlucky split can't decide the outcome. The decision metric is the WORST-fold Sharpe ratio across N_FOLDS=4 folds -- rewarding a config that doesn't fall apart in its worst quarter, not one that only shines in its best.

Bug found and fixed: the notebook printed a "winner" with no indication of whether it was actually distinguishable from the runner-up. A decidability check was added and simulation shows why it matters: with 4 folds and realistic fold-to-fold noise, EVERY candidate selector (min, mean, median of the folds) is close to a coin flip -- around 50-55% accuracy at correctly identifying a truly better config, with the ranking reordering on ~60% of reruns. Measured live on the shipped run: gap to runner-up = **+0.032 Sharpe** against a noise scale of **1.360** -- more than 40x smaller. This is printed now as `NOT DECIDABLE` rather than a false winner, and is the mechanical explanation for this project's earlier "config lottery", where three separate runs produced three different "winners" on effectively the same data.

Because of this, 'A: lean-cov' is retained on **structural, reproducible grounds** -- identifiability and rows-per-parameter (Section 2.3) -- and explicitly NOT because it won a Sharpe contest that cannot be won at this sample size.

8. The decline-labelling problem and the 12-day context window

A real user complaint: during a sustained multi-week decline, the chart showed bull-coloured bars partway down. Diagnosis: at the ORIGINAL windows, the LONGEST feature window in the whole set was 20 bars = 6.7 trading days -- so the detector's widest possible view of "context" was under a week. Inside any 6.7-day slice of a two-month decline there genuinely ARE bullish-looking bounces, so the detector was answering a shorter question than the chart was being judged on. Not a coding bug -- a timescale mismatch.

Measured on real daily NIFTY data, same fit, only the context-window reach changed (bars inside a decline = 20-day return ≤ –8%):

Context reach	Bear	Bull (the problem)	Sideways
~7 days (the old window)	90.7%	7.4%	1.9%
12 days (the fix, calendar-matched)	94.4%	0.0%	5.6%
20 days (measured saturation point)	100.0%	0.0%	0.0%

Fix applied: SWING_WIN and VOL_SLOW (drawdown and distance-from-MA) widened from 20 bars (6.7 days) to 36 bars (12 days). The momentum ladder (1/3/5 days) was deliberately left FAST -- lengthening every window uniformly would buy the same context at the cost of overall responsiveness; splitting them keeps the detector reactive while giving it something wider to be reactive WITHIN. Cost, stated plainly: a longer reach reacts more slowly -- inherent to the fix, not a defect.

This is a CONFIGURATION decision that changes emitted labels (SIDEWAYS occupancy shifts), unlike the six bugs in Section 9 which do not.

9. The six audited bugs (all fixed, labels re-verified after each)

A deliberate part-by-part audit of the whole generator, each part verified before moving to the next, with the shipped labels re-checked bit-identical to the v6 reference after every fix (since all six live in diagnostics or plotting, not the label-producing path itself).

#	Part	Bug	Fix
1	Direction bucketing	direction_buckets() gave every leftover state to the BULL bu[illegible]	[illegible]
2	Hysteresis	No guard existed against an inverted hysteresis band (exit s[illegible] which would silently make hysteresis a no-op.	[illegible]
3	VIX alignment	vix.ffill().bfill() -- after ffill, remaining NaNs are LEADING ba[illegible]	[illegible]
4	Config selection	An undecidable config ranking (Section 7) was printed as a [illegible]	[illegible] gap to the runner-up against the fo[illegible]
5	Chart shading	regime_blocks() ended each coloured band at its own last b[illegible]	[illegible]
6	Equivalence coverage	(Checked, not a bug) A prior fix pinned BAR_DIR_WEIGHT[illegible]	[illegible]

Also investigated and REFUTED as a bug hypothesis: excess SIDEWAYS occupancy was initially suspected to come from CONF_L or from the direction-bucketing rank scheme. Measurement showed CONF_L fires on 0.07% of bars (not the cause) and that DIRECTION_MODE='rank' is a bit-for-bit no-op versus the shipped rank scheme (0/2744 bars differ). The actual driver -- confirmed by ablation -- is BAR_DIR_WEIGHT interacting with CONF_L: blending toward per-bar momentum lowers the median winning bucket mass (0.693 → 0.564 at w=0.75), which then trips the 0.50 confidence floor far more often. A candidate fix (SIDE_TARGET_RATE, a fit-window-calibrated margin) exists and is measured in a SIBLING notebook (sideways_fix.ipynb) but is NOT part of the shipped master -- it changes emitted labels and was kept separate pending a decision.

10. Every chart in the master notebook

1. Free parameters / rows-per-parameter

Two bar charts: raw parameter count per config, and data-rows-per-parameter with a health-floor line at 10. Makes the capacity argument for 'A: lean-cov' visual.

2. Head-to-head, 4 panels

Worst-fold Sharpe (the decision metric) · full Sharpe spread with whiskers (shows the ranking is close) · params vs rows/param · switches per fold (churn).

3. Primary evaluation chart

THE main chart: black price line, background coloured by regime at every bar, blue dashed line marking the fit/test boundary.

4. Fold equity curves

4 consecutive time chunks, each showing cumulative return per config vs buy-and-hold -- reveals whether a bad fold is market-wide or config-specific.

5. Production-style view + VIX panel

Same regime-background chart at the shipped 70% fit fraction, with an India VIX panel underneath for volatility context.

6. Zoomed regime calls, 4 windows

The full series cut into four ~7-month slices at readable zoom -- includes the 2026 decline that was the original complaint window.

7. Forward-return validation

Box plots and mean-return bars per regime at 3/9/15-bar horizons -- the accountability check for whether labels predict anything. Reveals the L_BEAR/fwd_15 inconsistency (Section 6.4).

8. Confidence over time

Every bar's winning-direction confidence, coloured by label, with the CONF_L=0.5 override line and an H-reference line at 0.7.

9. H-vs-L decision plane

Scatter of trend_z vs trend_efficiency; only the top-left/top-right corners (strong AND clean) escalate to H. Visualises the zero-HMM intensity axis.

10. Run lengths + transition matrix

How long each regime typically lasts, plus a heatmap of what regime follows what (sanity check: bulls downgrade through L before flipping bear, never jump straight across).

11. Regime-driven long/flat vs buy-and-hold

A naive trading sense-check, not the product itself -- long only during BULL labels, flat otherwise, vs plain holding.

12. Fixes before/after

Fixes 1-3 off vs on, full series and zoomed into the single strongest sustained rally -- shows fewer, longer, more decisive regime calls.

13. FIX 4 attribution (BAR_DIR_WEIGHT)

Per-state (w=0) vs per-bar-blended (w=0.5) direction, full series and zoomed on a V-shaped reversal -- shows how little the blend weight actually changes.

14 & 15. The BAR_DIR_WEIGHT sweep

The most important honest result: predictive skill vs w with a seed-noise band (headroom check concludes UNMEASURABLE); and every w plotted in (descriptive-fidelity, predictive-skill) space with NO winner declared, since there is no defensible way to combine the two axes into one score.

11. Shipped configuration -- quick reference

Constant	Value	Note
Base config	v6 + 6 audited bug fixes + 12-day context window	
ADOPTED_CONFIG_NAME	'A: lean-cov'	N=5, diag covariance, 9 features, 114 params
BAR_DIR_WEIGHT	0.5	Direction = 50% HMM + 50% per-bar momentum
ENSEMBLE_K	4	Seeds [42,43,44,45]
INTENSITY_MODE	'frozen_z'	v6 value
ESCALATION_DURING_HOLD	'allow'	v6 value
DIRECTION_MODE	'rank'	Confirmed a bit-for-bit no-op vs the shipped scheme
CONF_L	0.50	Low-confidence → SIDEWAYS override
CONFIRM_BARS	2	Anti-flicker confirmation delay
Z_HI / EFF_HI	0.5 / 0.35	H-escalation entry thresholds
Z_HI_EXIT / EFF_HI_EXIT	0.35 / 0.25	H-escalation exit thresholds (hysteresis)
Momentum ladder	1 / 3 / 5 days	Unchanged -- kept fast deliberately
Context window (drawdown, dist_ma)	12 days (36 bars)	Widened from 6.7 days (20 bars)
EFF_WIN	9 bars (3 days)	
N_FOLDS / TRAIN_FRACTION	4 / 0.70	Walk-forward evaluation

12. Open items, carried forward

- L_BEAR's forward-15-bar return inversion (Section 6.4) -- mixed across folds, not resolved.
- SIDE_TARGET_RATE SIDEWAYS fix exists and is measured but not yet folded into the shipped master (changes emitted labels; needs a deliberate decision).
- The two-basin EM identifiability issue (Section 2.4) is a property of the feature set's collinearity, not fully resolved by any change made so far.
- Config ranking is fundamentally undecidable at this sample size (Section 7) -- more folds or more history would be needed to ever make it decidable, not a better selector.
- Vortex Indicator / intrabar High-Low features were considered and deliberately NOT added (the momentum-ladder + 12-day context change was chosen instead).

End of notes.